

Addressable Recall Compaction for Long Context-Window Control in AI Agents

Thang Dang^{1*†}, Yuma Ichikawa^{2,3*†}, Sakina Fatima¹, Koichi Shirahata²

¹Fujitsu Research of America; ²Fujitsu Limited Japan; ³RIKEN Center for AIP
tdang@fujitsu.com, ichikawa.yuma@fujitsu.com, sakina.fatima@fujitsu.com, k.shirahata@fujitsu.com

Abstract

Long-horizon LLM agents accumulate reasoning traces, actions, and tool observations that can eventually exceed a model’s fixed context window. Existing compaction methods address this limitation by discarding, summarizing, or retrieving earlier information, but they may remove task-critical details or fail to recover them reliably. We propose **ARC** (Addressable Recall Compaction), a context-management framework that separates archival storage from active-context presentation. ARC stores tool observations in an append-only, ID-addressable log and replaces older observations with compact citations when compaction is required. The agent can subsequently use these identifiers to request stored content without re-executing the corresponding tools or depending solely on similarity-based retrieval. We evaluate ARC using Qwen3-8B with a 16k context window and Qwen3-32B with a 32k context window. On the Needle-in-a-Haystack evaluation, ARC achieves an average exact-answer accuracy of 99.40%, compared with 88.12% for the best-performing baseline in our evaluation. ARC also reduces estimated serving time and HBM traffic under our hardware-cost model. On the LongBench-v2 Hard subset, ARC obtains an average accuracy of 29.97%, compared with 28.25% for the best-performing baseline. These results indicate that explicit, address-based recall can improve information retention and serving efficiency relative to the evaluated context-management baselines under the tested settings.

Introduction

In recent years, the paradigm of Large Language Models (LLMs) (Masterman et al. 2024) has transitioned from passive text generation (Li et al. 2024) to active, autonomous agency. By embedding an LLM within an interactive execution loop, modern AI agents (Durante et al. 2024) can perceive environments, reason over objectives, and execute actions via external tools, APIs, or bash shells. At each discrete time step t , the agent reads the cumulative historical transcript of the trajectory, synthesizes an internal monologue, and emits a structured action. The underlying runtime environment executes this action and appends the resulting observation back into the transcript, preparing the agent for the subsequent turn. Because contemporary autoregressive

language models remain fundamentally stateless between independent API calls or inference passes, this monotonically growing transcript serves as the agent’s singular, monolithic working memory.

However, this architecture introduces a severe engineering and algorithmic bottleneck: the context-window limit (Tang et al. 2025). Every LLM is bounded by a strict maximum sequence length that it can process at once. Because the agentic trajectory at any given turn is a literal concatenation of all prior prompts, tool outputs, and environment responses, the transcript grows continuously and without bound as the agent keeps acting.

Once the cumulative transcript outgrows this maximum capacity, the system encounters an unrecoverable failure state: the LLM can no longer process the next step, effectively paralyzing the agent. For long-horizon planning (Erdogan et al. 2025), complex software engineering tasks, or open-ended web browsing, hitting this context wall is not a matter of if, but when rendering even modern ultra-large context windows insufficient.

To prevent this catastrophic overflow, contemporary agent frameworks rely on compaction or compression (Ichikawa et al. 2026): a suite of lossy transformation techniques designed to compress an over-long transcript into a truncated form that fits comfortably within context-window length. The current state-of-the-art methods generally fall into four paradigms:

- **Naive Truncation** (Wang et al. 2026): Dropping the oldest N turns entirely, assuming older context is irrelevant.
- **Hierarchical Summarization** (Kang et al. 2026): Periodically prompting the LLM to condense historical turns into high-level textual summaries.
- **State Folding** (Fu et al. 2026): Mapping conversational histories into explicit, structured JSON or key-value state objects.
- **Retrieval-Augmented Generation (RAG)** (Park et al. 2023): Offloading historical context into an external vector database and retrieving relevant chunks via semantic similarity.

All four paradigms are fundamentally lossy: once history is pruned, paraphrased, or vectorized, exact recovery is no longer guaranteed. If the discarded content contains a critical *needle* such as a variable name, error string, or user

*These authors contributed equally.

†Corresponding author.

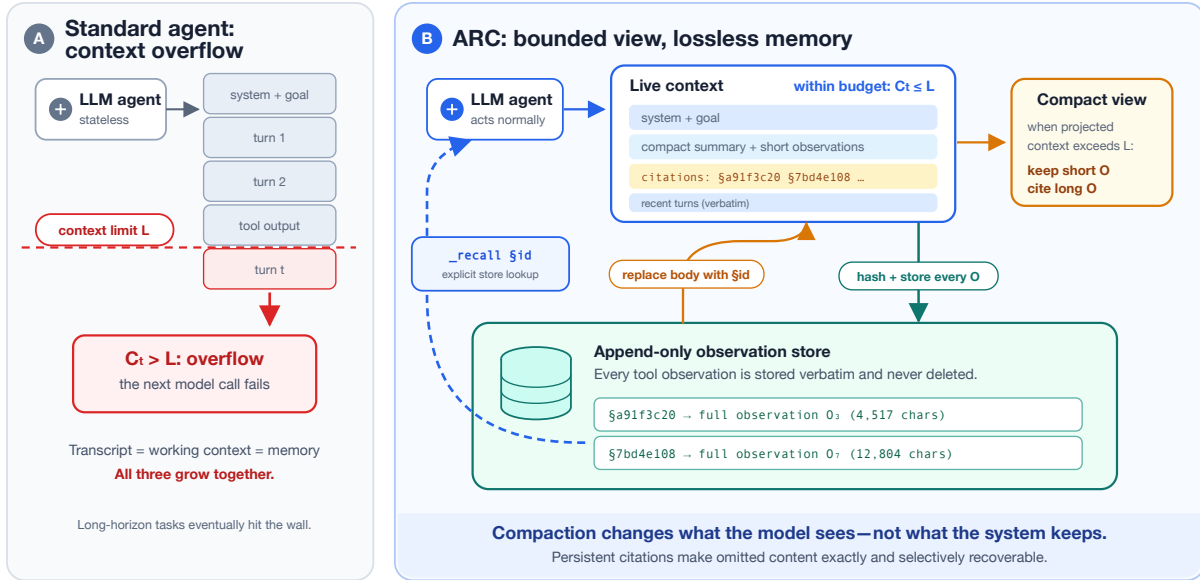


Figure 1: Our Addressable Recall Compaction - ARC vs a standard agent context management.

constraint, the agent must either re-derive it at high cost or hallucinate a substitute, compounding errors over long trajectories.

We address this limitation by asking whether transcript compaction can remain strictly lossless while keeping the active prompt bounded. We introduce **ARC** (Addressable Recall Compaction) (Fig. 1), which decouples archival storage from in-context representation. ARC maintains (1) an append-only **Addressable Store** that preserves historical turns and tool outputs verbatim, and (2) a **Bounded-Size Active View** that replaces older content with explicit pointers into that store. With recall tools that resolve these pointers on demand, ARC preserves access to any historical *needle* without exceeding the active context budget.

Summary of Contributions

- We characterize how lossy compaction fails in long-horizon LLM agent trajectories.
- We present ARC, a lossless compaction framework based on addressable pointers and on-demand recall.
- We show that ARC prevents context-overflow failures and outperforms summarization- and RAG-based baselines on long-context, tool-heavy tasks.

Related Works

Prior compaction methods differ in what they remove, when removal happens, and whether discarded content can later be recovered. We organize them by this last property, since it determines whether compaction is *safe*: whether task-critical information can survive the process.

Full-Context (No Compaction). Some agent systems simply append every turn to the transcript indefinitely, as in ReAct and Reflexion (Yao et al. 2023; Shinn et al. 2023). This preserves perfect fidelity while context remains available,

but it is not a true compaction strategy: once $|T| > L$, the method fails.

Sliding Window. A common baseline keeps only the most recent N turns and drops older ones (Ding et al. 2026). This is simple and budget-compliant, but once a value falls outside the window, it is lost permanently.

LLM Summary. Another approach replaces older turns with a natural-language summary produced by the model (Packer et al. 2023). This retains more semantic content than truncation, but remains lossy: omitted or paraphrased details cannot be recovered, and each compaction event incurs extra latency and cost.

Structured State. Instead of free text, prior work may compress history into a fixed-schema state object such as JSON (Fu et al. 2026). Schemas can preserve anticipated fields more reliably than open-ended summaries, but any unmodeled information is still discarded irreversibly.

RAG Memory. Retrieval-based methods store past observations externally and re-inject retrieved chunks into context when needed (Lewis et al. 2020). Although the store may retain everything, recall is only implicit: the agent cannot request a specific prior item directly, and recovery depends on retrieval quality.

Summary. Across these approaches, the key limitation is the absence of reliable, addressable recall. Some methods discard information outright; others preserve it only indirectly through summaries, schemas, or similarity-based retrieval. ARC addresses this gap by storing observations verbatim under canonical identifiers and replacing them in the live transcript with short citations, allowing explicit recovery of exact prior content on demand.

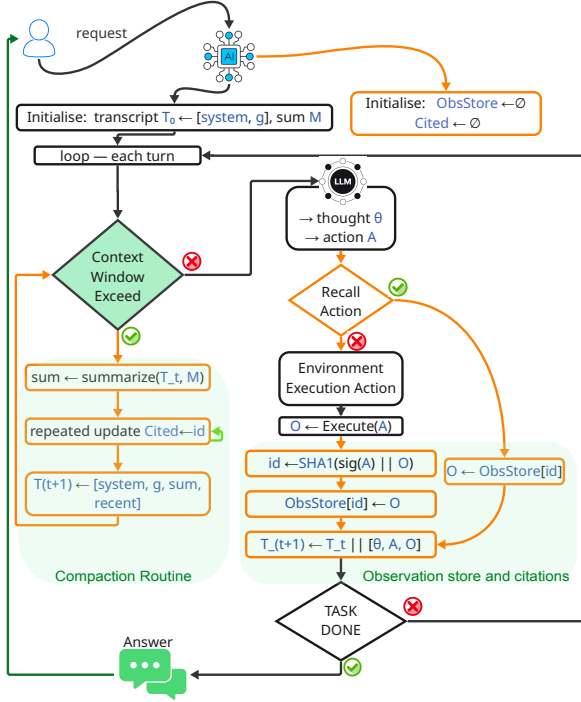


Figure 2: Our Addressable Recall Compaction workflow.

Background

A long-horizon LLM agent operates in a loop: at each turn it receives an observation (a tool result, an error message, a user message), produces a thought and an action, and appends all of it to a growing transcript $T_t = (o_1, a_1, o_2, a_2, \dots, o_t, a_t)$ that is re-sent to the model on every subsequent call (Yao et al. 2023). The model itself is stateless between calls; everything it “remembers” about the task is whatever fits in T_t .

This creates a hard constraint. Every model serves requests under a fixed context budget L (measured in tokens). Once $|T| > L$, the next call fails outright, producing a context-window overflow, regardless of how important the content being dropped is. Overflow is not a rare edge case: any task that runs long enough (many tool calls, large tool outputs, extended reasoning) will eventually exceed L , so a real agent must have *some* answer to this moment.

We call the general problem **compaction**: producing a shorter transcript T'_t from T_t once $|T| > L$, such that the agent can continue operating within budget. Every method surveyed in this paper is a compaction strategy in this sense, and they differ along one central axis: what happens to the information that no longer fits verbatim in T'_t . We say a value is **recoverable** under a given strategy if there exists some later point in the task at which the agent can retrieve that value’s exact original form; otherwise it is **lost**. A compaction strategy is *lossy* if it can produce T'_t from which some value is unrecoverable by construction (e.g., truncated away, paraphrased, or never stored outside the live transcript). This distinction of lost versus recoverable, and if recoverable, by what mechanism the agent actually retrieves the value, is

the organizing axis for the related work below, since it determines whether a compaction event can silently destroy information the agent will later need, independent of how fluent or efficient the compaction itself is.

To measure this directly, we adopt a needle-in-a-haystack (Nelson et al. 2024; Kamradt 2023) protocol: a single random, unforgeable value (the needle) is introduced early in a task, the agent is driven through one or more forced compaction events, and the task is scored by whether the agent can still produce the needle’s exact value when asked. This isolates recoverability from general task competence, since a method can compact skillfully and still fail this test if the mechanism it uses provides no reliable path back to the needle once it has left the live transcript.

Proposed Method

Design Principle

Every baseline in the Related Works Section conflates two decisions that need not be coupled: (1) what to keep, and (2) what to show the model right now. ARC separates them. It keeps *everything*, always, in a content-addressed store; it decides what to show using the same length-bounded logic as the other baselines, but every omission leaves behind an explicit, dereferenceable pointer. Recovery is then a decision that the agent makes, not a probability that the retriever gets right.

ARC Algorithm (Fig. 2)

Observation store and citations. For every normal action A , ARC stores the resulting observation O under a stable id computed as $\text{Hash}(\text{signature}(A), O)$: concretely, SHA1 over the action signature, an ASCII unit-separator byte, and the observation. The visible id uses the first 8 hex characters and is extended in 4-character increments only if two different observations collide. Each `ObsStore` record keeps the full content, command signature, fixed head/tail previews, length metadata, return code, creation step, and recall bookkeeping. The store is append-only for the lifetime of the task.

When compaction removes a long observation from the visible transcript, ARC leaves a citation `$id` containing its fixed head preview, tail preview, metadata, and a `_recall $id` hint. Observations shorter than ρ remain verbatim. The set `Cited` enforces citation persistence: once an id has appeared as a recall handle, later compactions preserve that handle rather than re-summarizing or renaming it.

Recall action. `_recall $id` is intercepted by the agent framework before it reaches the environment. A valid recall injects `ObsStore[id]` back into the transcript; an invalid id returns a nearest-match suggestion instead of failing silently. Recalls are controlled by a per-step limit r (default $r = 2$), a total materialized-recall budget, and a per-recall maximum length; least-recently-used recalled bodies are evicted back to citation stubs when the budget is exceeded. A lighter `_recall_meta $id` action restores only citation-level metadata.

Compaction routine. Summarize is deterministic and does not call an LLM, unlike the LLM-summary method,

this step adds no extra model invocation or latency. It truncates older thoughts, keeps short observations inline, replaces longer observations by citations, and retains the most recent turns verbatim. If the assembled summary still exceeds M , older already-cited entries are collapsed from full citation stubs to bare `$id` references. These operations change only the active view; the full bodies remain recoverable from `ObsStore`.

Triggering. ARC supports both proactive and reactive triggering. In proactive mode, the agent estimates prompt length before each model call and compacts before overflow. In our real-LLM evaluation, we use the reactive path: compaction occurs after a context-window-overflow error, with fixed call indices used to isolate post-compaction recovery from stochastic overflow timing. Both modes invoke the same deterministic compaction routine and recall mechanism.

The key invariant is that after compaction, the visible transcript has a bounded size $K \leq L$ independent of the number of elapsed turns, while `ObsStore` retains every observation at full fidelity. ARC, therefore, shrinks the model’s view without shrinking the task’s ground truth.

System-Level Guarantee

The guarantee we establish for ARC is a property of the memory mechanism, not of the agent that uses it. It ensures that any stored observation remains exactly recoverable through a valid address, but it does not ensure that the language model will choose the correct address or reason accurately once the content is retrieved. We therefore draw a deliberate distinction between address-conditioned recoverability, which the mechanism provides, and end-to-end task success, which the agent must additionally achieve. In the same spirit, we use persistent addressability to mean that every issued address remains valid in the external catalog for the lifetime of the task. This is a weaker and more realistic requirement than keeping all citations simultaneously visible in the active prompt, which, as we show in the appendix, is incompatible with a turn-independent context bound.

Let L denote the usable input-token budget, defined as the context limit minus the capacity reserved for the model’s maximum completion length. Within the active view, ARC assigns a fixed token budget to each of the five components: the immutable task prefix, the deterministic summary, the recent suffix, a single page of the external citation catalog, and the recalled content. Let K be the sum of these budgets, and require $K \leq L$. Crucially, ARC never recovers a large observation through head–tail truncation, which would irreversibly discard interior tokens; instead, it returns the observation through exact, non-overlapping recall chunks of at most q payload tokens each, ensuring that the full content is always reconstructible on demand.

Theorem 1 (Informal ARC guarantee). *Suppose ARC (i) assigns injective, fixed-width occurrence identifiers up to a declared task capacity, (ii) stores every tool observation verbatim before any compaction can remove it, (iii) keeps the complete citation catalog in external memory while materializing only a token-bounded page in the prompt, and (iv) enforces the active-view budget $K \leq L$ before every model invocation. Then, at every turn, the model input contains at most*

K tokens, independently of how many turns have elapsed. Moreover, for every stored observation O with valid identifier id , ARC returns the exact archived token sequence without re-executing the original action; if $|O| > q$, the full sequence is reconstructed from exactly $\lceil |O|/q \rceil$ non-overlapping recall responses, each carrying at most q payload tokens. Consequently, ARC is observation-lossless at the system level while maintaining a uniformly bounded active view.

We emphasize that this guarantee is intentionally conditioned on a valid address: it establishes exact availability and bounded presentation, but not that the model will request the correct identifier. The appendix provides the self-contained definitions and full proofs. It further shows that this design overhead is essentially unavoidable: any method that supports the exact recovery of arbitrary histories while keeping the active state bounded must use external memory that grows linearly in the worst case. ARC’s append-only payload growth, therefore, matches this lower bound to first order rather than reflecting a limitation of our particular implementation.

Experimental Setup

Models Configuration

We evaluate Qwen3-8B and Qwen3-32B served with vLLM, using 16k and 32k context windows respectively and a 4k completion cap. The context window is the vLLM limit on prompt plus completion length; when a trajectory exceeds it, vLLM raises a context-window error and the agent invokes its compaction path. Thus each method is compared under the same model scale, context budget, and per-call generation cap.

Benchmarks

Evaluation metrics. We report accuracy and, separately, `no_ans`: the number of trials with no accepted submission. A submission counts only after the harness protocol is completed; premature, blocked, crashed, or missing submissions are counted as failures. Since non-submissions are already incorrect, `no_ans` is diagnostic rather than a separate success metric: it distinguishes wrong answers from failures to reach an answer at all.

HBM bandwidth and roofline metric (Austin et al. 2025). Context management changes not only task success but also serving cost. We therefore replay each method’s recorded prompt/completion trajectory through a fixed hardware model of one NVIDIA H200 143GB GPU and estimate prefill, decode, and total time. During autoregressive decode, each generated token streams the model weights M_w , amortized over the concurrent batch size N_b , plus the request’s own KV cache of length L :

$$\text{bytes per decoded token (per request)} = M_w/N_b + L \cdot \kappa, \quad (1)$$

where κ is the per-token KV-cache footprint. Decode time is bytes streamed divided by HBM bandwidth, while prefill time is prefill FLOPs divided by peak Tensor Core throughput. We report $T_{\text{total}} = T_{\text{prefill}} + T_{\text{decode}}$ as a roofline estimate of serving cost.

Method	Model	Ctx.	Max-T.	Pass ↑ / Attempts	Mean No-Ans↓	Mean Pass↑	Mean Acc↑	SD Acc	Mean ± SD
Full_context	Qwen3-8B	16k	4096	0 / 933	311	0	0.0%	0.0%	0.0% ± 0.0%
Sliding_window		—	—	219 / 933	48.67	73	23.47%	1.15%	23.47% ± 1.15%
LLM_summary		—	—	241 / 933	33	80.33	25.83%	0.96%	25.83% ± 0.96%
Structured_state		—	—	231 / 933	40	77	24.73%	1.94%	24.73% ± 1.94%
RAG_memory		—	—	229 / 933	45	76.33	24.53%	2.70%	24.53% ± 2.70%
ARC		—	—	256 / 933	16.33	85.33	27.47%	1.69%	27.47% ± 1.96%
Full_context	Qwen3-32B	32k	4096	0 / 933	311	0	0%	0.0%	0.0% ± 0.0%
Sliding_window		—	—	286 / 933	23.67	95.33	30.67%	0.68%	30.67% ± 0.68%
LLM_summary		—	—	233 / 933	73.67	77.67	24.97%	2.28%	24.97% ± 2.28%
structured_state		—	—	282 / 933	27	94	30.2%	0.30%	30.2% ± 0.30%
RAG_memory		—	—	256 / 933	58	85.33	27.43%	0.75%	27.43% ± 0.75%
ARC		—	—	303 / 933	7.33	101	32.47%	1.42%	32.47% ± 1.42%

Table 1: LongBench-v2 hard-subset accuracy, 3 seeds (42, 82, 122) of 311 tasks each (933 attempts/method).

Method	Model	Ctx.	Max-T.	Pass↑ / Attempts	Mean No-Ans↓	Mean Pass↑	Mean Acc↑	SD Acc	Mean ± SD
Full_context	Qwen3-8B	16k	4096	1104 / 3000	631.67	368.00	36.80%	2.59%	36.80% ± 2.59%
Sliding_window		—	—	1686 / 3000	437.33	562.00	56.20%	0.90%	56.20% ± 0.90%
LLM_summary		—	—	2153 / 3000	279.33	717.67	71.77%	1.36%	71.77% ± 1.36%
Structured_state		—	—	2045 / 3000	314.33	681.67	68.17%	1.64%	68.17% ± 1.64%
RAG_memory		—	—	2387 / 3000	201.00	795.67	79.57%	0.80%	79.57% ± 0.80%
ARC		—	—	2970 / 3000	9.67	990.00	99.00%	0.17%	99.00% ± 0.17%
Full_context	Qwen3-32B	32k	4096	88 / 3000	964.67	29.33	2.93%	0.76%	2.93% ± 0.76%
Sliding_window		—	—	2860 / 3000	44.33	953.33	95.33%	0.64%	95.33% ± 0.64%
LLM_summary		—	—	2817 / 3000	22.33	939.00	93.90%	0.52%	93.90% ± 0.52%
Structured_state		—	—	2899 / 3000	15.33	966.33	96.63%	0.78%	96.63% ± 0.78%
RAG_memory		—	—	2900 / 3000	12.33	966.67	96.67%	1.15%	96.67% ± 1.15%
ARC		—	—	2994 / 3000	0.67	998.00	99.80%	0.17%	99.80% ± 0.17%

Table 2: Needle-in-a-haystack Task, 3 seeds (42, 82, 122) of 1000 tasks each (3000 attempts/method).

The attainable batch size is limited by the HBM remaining after model weights and a fixed scratch reserve S :

$$N_b = \left\lfloor \frac{n_{\text{gpu}} \cdot \text{HBM}_{\text{gpu}} - M_w - S}{\bar{L} \cdot \kappa} \right\rfloor, \quad (2)$$

where $\bar{L}$ is the decode-token-weighted mean context length. Shorter active contexts therefore reduce the KV term directly and increase N_b , reducing the amortized weight term M_w/N_b . In the Results section, we abbreviate roofline quantities as **t_tol.** (total time), **t_dec.** (decode time), **t_pre.** (pre-fill time), **bw** (bandwidth), and **Sav.** (savings).

Paired significance testing. Because all methods use the same seeded tasks/needles, outcomes are paired. For each ARC-vs-baseline comparison, we use McNemar’s test on the discordant cells a_{only} (ARC succeeds, baseline fails) and b_{only} (baseline succeeds, ARC fails), reporting the continuity-corrected statistic

$$\chi^2 = \frac{(|a_{\text{only}} - b_{\text{only}}| - 1)^2}{a_{\text{only}} + b_{\text{only}}}, \quad (3)$$

and an exact binomial p -value over discordant pairs. We apply this test to correctness (**success** vs. **failure**) and to completion (**accepted submission** vs. **no_ans**), separating answer quality from the ability to finish the protocol.

LongBench-v2 (Hard Subset) We evaluate on the LongBench-v2 hard subset (Bai et al. 2025), containing 311 tasks across single-document QA, multi-document QA, long

in-context learning, structured-data understanding, long-dialogue-history understanding, and code-repository understanding. Unlike the needle benchmark, LongBench-v2 requires multi-step reasoning over long inputs, so it tests whether ARC’s recall mechanism helps beyond verbatim retrieval.

Needle-in-a-Haystack (NIH) (Gao et al. 2026) The benchmark needs a single, unambiguous, unforgeable fact that the agent must retain across compaction events. That fact is the needle, a 24-character token drawn from a 30-character alphabet, embedded inside a synthetic file as `secrets.txt`.

Needle benchmark guardrail. NIH evaluations can be confounded by self-doubt (Gao et al. 2026): a model may find a random token but hesitate to submit it. Our harness reduces this risk by making the needle a banner-delimited instruction labeled THE NEEDLE FOR THIS TASK, marking distractors as noise, and scoring only the submitted NEEDLE=<value> token rather than surrounding prose. All methods also use the same seeded needle pool, so residual per-needle difficulty is paired across methods.

Results

We evaluate ARC on the LongBench-v2 hard subset with Qwen3-8B and Qwen3-32B. As shown in Table 1, ARC achieves the best accuracy at both scales: 27.47% on 8B and

32.47% on 32B, improving over the strongest baselines by +1.64 and +1.80 points, respectively.

ARC also sharply reduces unanswered cases, with mean No-Answer rates of 16.33 on 8B and 7.33 on 32B, while ‘Full_context’ fails completely at both context limits. Unlike static compression methods, ARC scales favorably to the larger backbone and remains stable across seeds.

Table 3 shows that ARC’s gains are accompanied by markedly lower HBM traffic. On LongBench, ARC reduces bandwidth by 38.8% on Qwen3-8B and 73.5% on Qwen3-32B relative to ‘Sliding_window’, corresponding to substantially lower wall-clock time. ARC is the only method that improves task success while also reducing memory traffic and decoding cost.

Method	t_tot. (s) ↓	t_dec. (s) ↓	t_pre. (s) ↓	bw ↓	Sav. (%) ↑
Qwen3-8B					
Sliding_window	6.53	5.76	0.77	27.63	0.0
LLM_summary	8.11	7.47	0.64	35.84	-24.3
Structured_state	9.68	9.07	0.61	43.53	-9.8
RAG_memory	14.68	13.85	0.83	66.48	-124.9
ARC	3.99	3.32	0.67	15.95	38.8
Qwen3-32B					
Sliding_window	30.57	28.76	1.81	138.05	0.0
LLM_summary	75.5	72.45	3.05	347.73	-146.9
Structured_state	28.67	27.25	1.42	130.79	6.2
RAG_memory	77.01	74.55	2.46	357.83	-151.9
ARC	8.11	6.44	1.68	30.9	73.5

Table 3: High-Bandwidth Memory (HBM) comparison on success submitted Longbench tasks.

Method	t_tot. (s) ↓	t_dec. (s) ↓	t_pre. (s) ↓	bw ↓	Sav. (%) ↑
Qwen3-8B					
Sliding_window	6.38	5.77	0.61	27.70	0.0
LLM_summary	9.49	8.95	0.54	42.97	-48.9
Structured_state	10.59	10.12	0.47	48.55	-66.0
RAG_memory	6.47	6.03	0.44	28.92	-1.4
ARC	1.26	0.01	0.24	4.87	80.3
Qwen3-32B					
Sliding_window	0.83	0.58	0.26	2.76	0.0
LLM_summary	0.85	0.64	0.2	3.09	-77.4
Structured_state	0.67	0.51	0.16	2.46	33.1
RAG_memory	0.64	0.47	0.17	2.25	11.9
ARC	0.55	0.33	0.22	1.59	59.3

Table 4: High-Bandwidth Memory (HBM) comparison on success submitted Needle-in-a-haystack tasks.

To rigorously assess whether the performance gains of our proposed method, ARC, are statistically significant, we conduct a McNemar’s test on the matched-paired binary outcomes across all 933 evaluation attempts (311 tasks $\times$ 3

seeds). McNemar’s test focuses exclusively on the discordant pairs—representing the cases where only one of the two compared methods succeeds. The results are detailed in Table 5. On Qwen3-8B, ARC yields odds ratios from 2.22 against ‘LLM_summary’ to 3.37 against ‘Sliding_window’, showing a clear advantage on discordant cases. The gap widens on Qwen3-32B: ARC reaches odds ratios of 10.48 against ‘LLM_summary’ and 8.24 against ‘RAG_memory’, while still outperforming the strongest larger-model baselines, ‘Sliding_window’ and ‘Structured_state’, with odds ratios of 3.58 and 3.95. Since ‘Full_context’ records zero successes at both scales, ARC also attains infinite odds ratios in those comparisons.

Table 2 shows near-perfect retrieval performance for ARC on Needle-in-a-Haystack. ARC reaches 99.00% accuracy on Qwen3-8B and 99.80% on Qwen3-32B, exceeding the strongest baseline, ‘RAG_memory’, by 19.43 and 3.13 points, respectively, while nearly eliminating unanswered queries.

ARC also has the lowest variance across seeds (0.17% for both models). In contrast, ‘Full_context’ remains unreliable even with a 32k window, reaching only 2.93% on Qwen3-32B, indicating that larger context alone does not solve precise retrieval.

Table 6 confirms that these retrieval gains are statistically significant in every paired comparison ($p \ll 0.001$). On Qwen3-8B, ARC dominates all baselines, including ‘RAG_memory’ (599 ARC-only wins vs. 25 baseline-only wins; odds ratio 23.96). On Qwen3-32B, ARC remains significantly better despite the stronger baselines, with an odds ratio of 18.5 against ‘RAG_memory’ and an infinite odds ratio against ‘Full_context’.

Table 4 shows that ARC improves efficiency as well as retrieval quality. On Qwen3-8B, ARC raises successful retrievals to 991 while reducing total inference time from 6.47 s (‘RAG_memory’) to 1.26 s and cutting HBM traffic by 80.3%. On Qwen3-32B, ARC again achieves the highest success count (999), the lowest latency (0.55 s), and the lowest bandwidth use (1.59 TB). These results indicate that ARC improves accuracy without the usual latency-bandwidth tradeoff.

Overall, these results show that ARC improves both retrieval effectiveness and computational efficiency. Unlike existing memory management methods that often trade latency for accuracy, ARC simultaneously achieves near-perfect retrieval performance while significantly reducing decoding cost and HBM bandwidth requirements, making it well suited for efficient long-context inference.

Discussion and Limitations

Why the margin narrows on LongBench-v2. Needle-in-a-haystack isolates verbatim recall—exactly what ARC targets—while LongBench-v2 hard tasks additionally require synthesizing information across a long document, so perfect recall can still yield a wrong answer through faulty reasoning. On LongBench-v2 (Table 1), ARC still leads every baseline at both scales (27.47% at Qwen3-8B, 32.47% at Qwen3-32B), but the margin over the closest competitor narrows to 1.6–2.3 points (LLM_summary at 8B, sliding_window at

A	B	A-Only	B-Only	Both	Neither	χ^2 p-val	Exact p-val	Odds Ratio	Per-Seed Odds ($\pm$ SD)	Dis. Seeds
Qwen3-8B										
ARC	Full_context	884	0	0	49	0	7.75×10^{-267}	inf	inf	0
ARC	LLM_summary	91	41	793	8	2.0×10^{-5}	8.0×10^{-6}	2.22	2.5 ± 1.38	3
ARC	RAG_memory	123	37	761	12	0	2.73×10^{-12}	3.32	3.88 ± 2.21	3
ARC	Sliding_window	138	41	746	8	0	8.88×10^{-14}	3.37	3.97 ± 2.2	3
ARC	Structured_state	113	42	771	7	0	5.24×10^{-9}	2.69	3.27 ± 2.01	3
Qwen3-32B										
ARC	Full_context	911	0	0	22	0	5.78×10^{-275}	inf	inf	0
ARC	LLM_summary	220	21	691	1	0	2.63×10^{-43}	10.48	10.57 ± 1.03	3
ARC	RAG_memory	173	21	738	1	0	3.19×10^{-31}	8.24	8.27 ± 0.56	3
ARC	Sliding_window	68	19	843	3	0	6.18×10^{-8}	3.58	3.71 ± 1.12	3
ARC	Structured_state	79	20	832	2	0	8.96×10^{-10}	3.95	4.07 ± 1.05	3

Table 5: McNemar test on Longbench task. Number of Pairs (hard task) = 311x3 seeds (42, 82, 122).

A	B	A-Only	B-Only	Both	Neither	χ^2 p-val	Exact p-val	Odds Ratio	Per-Seed Odds ($\pm$ SD)	Dis. Seeds
Qwen3-8B										
ARC	Full_context	1882	16	1089	13	0	0	117.62	123.43 ± 30.41	3
	LLM_summary	831	22	2140	7	0	3.5×10^{-214}	37.77	40.32 ± 13.76	3
	RAG_memory	599	25	2372	4	0	4.5×10^{-144}	23.96	26.42 ± 10.13	3
	Sliding_window	1300	17	1671	12	0	0	76.47	89.72 ± 45.54	3
	Structured_state	934	20	2037	9	0	8.8×10^{-247}	46.7	52.87 ± 21.05	3
Qwen3-32B										
ARC	full_context	2892	0	106	2	0	0	inf	inf	0
	LLM_summary	67	2	2931	0	0	4.09×10^{-18}	33.5	9.5 ± 0.0	1
	RAG_memory	37	2	2961	0	0	1.42×10^{-9}	18.5	5.5 ± 0.0	1
	Sliding_window	133	2	2865	0	0	2.11×10^{-37}	66.5	20.0 ± 0.0	1
	Structured_state	46	2	2952	0	0	4.18×10^{-12}	23.0	9.5 ± 0.0	1

Table 6: McNemar test on Needle-in-a-haystack task. Number of Pairs = 1,000x3 seeds (42, 82, 122).

32B), versus 4–7.5 points over the weaker baselines. ARC’s advantage is therefore better interpreted as a lower bound on how much of LongBench’s failure rate is attributable to recall, further corroborated by its markedly lower no-answer rate (16.33 and 7.33 out of 311, versus 23–74 for every baseline)—rather than the full ceiling of ARC’s benefit.

Statistical significance. Per-trial McNemar tests (Tables 6, 5) confirm ARC significantly outperforms every baseline on both benchmarks and both model scales ($p < 0.05$). Against ‘Full_context’, ARC wins every one of the 933 paired trials at both 8B and 32B, with zero baseline-only wins. Against the remaining baselines, b_only counts (baseline succeeds, ARC fails) stay small relative to a_only 37–42/933 at 8B and 19–21/933 at 32B—yielding odds ratios from 2.2 up to 10.5 in ARC’s favor; the effect is strongest for ‘LLM_summary’ at 32B (odds ratio 10.48) and weakest, though still significant, for ‘Sliding_window’ and ‘Structured_state’, whose closer overall accuracy reflects that reasoning errors, unlike recall failures, can occasionally favor a baseline over ARC on individual tasks.

Scope and overhead. All experiments use the Qwen3 model family; generalization to other tokenizers, reasoning styles, or instruction-following behavior for the _recall convention remains open. Each ARC citation (head preview, tail preview, recall hint) also adds a small, fixed token overhead relative to discarding an observation outright; quantifying this overhead and its effect on the effective context budget

is reported in the supplementary document.

Conclusion

Lossy compaction trades context-window survival for a chance of losing exactly the fact an agent needs later, and every mainstream compaction strategy makes this trade. ARC shows the trade is not necessary: by separating an append-only, content-addressed store of everything the agent has seen from a bounded-size, citation-annotated view of it, compaction becomes reversible without sacrificing the constant-size guarantee that made truncation attractive in the first place. Across two representative configurations spanning both model sizes (Qwen3-8B/32B, both at a 4,096-token completion cap), this closes nearly all of the recovery gap on a benchmark built to isolate recall failure (99.40% vs. 88.12% for the best prior method) and yields a smaller but consistent gain on a benchmark that also demands reasoning (29.97% vs. 28.25%). The main open questions are quantifying ARC’s own token overhead, testing generalization beyond the Qwen3 family, extending the evaluation to the remaining points of the model/context-window/max-token grid, and understanding why implicit retrieval still wins on dialogue-history tasks — a signal that explicit and implicit recall are complementary rather than competing mechanisms.

References

- Austin, J.; Douglas, S.; Frostig, R.; Levskaya, A.; Chen, C.; Vikram, S.; Lebron, F.; Choy, P.; Ramasesh, V.; Webson, A.; and Pope, R. 2025. How to Scale Your Model. Retrieved from <https://jax-ml.github.io/scaling-book/>.
- Bai, Y.; Tu, S.; Zhang, J.; Peng, H.; Wang, X.; Lv, X.; Cao, S.; Xu, J.; Hou, L.; Dong, Y.; et al. 2025. Longbench v2: Towards deeper understanding and reasoning on realistic long-context multitasks. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 3639–3664.
- Ding, X.; Sun, R.; Zhang, Y.; Yan, X.; Zhou, Y.; Huang, K.; Fu, S.; Aviles-Rivero, A. I.; Xie, C.; and Zhu, Y. 2026. Sliding-Window Merging for Compacting Patch-Redundant Layers in LLMs. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, 20826–20834.
- Durante, Z.; Huang, Q.; Wake, N.; Gong, R.; Park, J. S.; Sarkar, B.; Taori, R.; Noda, Y.; Terzopoulos, D.; Choi, Y.; et al. 2024. Agent ai: Surveying the horizons of multimodal interaction. *arXiv preprint arXiv:2401.03568*.
- Erdogan, L. E.; Lee, N.; Kim, S.; Moon, S.; Furuta, H.; Anumanchipalli, G.; Keutzer, K.; and Gholami, A. 2025. Plan-and-act: Improving planning of agents for long-horizon tasks. *arXiv preprint arXiv:2503.09572*.
- Fu, J.; Chen, J.; Wang, M.; He, A.; Sheng, M.; Ke, X.; Zhu, Y.; and Gao, Y. 2026. VikingMem: A Memory Base Management System for Stateful LLM-based Applications. *arXiv preprint arXiv:2605.29640*.
- Gao, Y.; Xiong, Y.; Wu, W.; Li, B.; Zhong, Y.; and Wang, H. 2026. U-niah: Unified rag and llm evaluation for long context needle-in-a-haystack. *ACM Transactions on Information Systems*, 44(3): 1–30.
- Ichikawa, Y.; Kimura, K.; Yoshida, A.; Fujimoto, Y.; Tokura, H.; Arai, Y.; Ishii, Y.; Kawakami, Y.; Shikada, G.; Jacquemond, A.; et al. 2026. OneComp: One-Line Revolution for Generative AI Model Compression. *arXiv preprint arXiv:2603.28845*.
- Kamradt, G. 2023. Needle In A Haystack: Pressure-testing LLM Long-Context Retrieval. <https://github.com/gkamradt/needle-in-a-haystack>. GitHub repository.
- Kang, M.; Chen, W.-N.; Han, D.; Inan, H. A.; Wutschitz, L.; Chen, Y.; Sim, R.; and Rajmohan, S. 2026. Acon: Optimizing context compression for long-horizon llm agents, 2025. *URL https://arxiv.org/abs/2510.00615*.
- Lewis, P.; Perez, E.; Piktus, A.; Petroni, F.; Karpukhin, V.; Goyal, N.; Küttler, H.; Lewis, M.; Yih, W.-t.; Rocktäschel, T.; Riedel, S.; and Kiela, D. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems*, volume 33, 9459–9474.
- Li, J.; Tang, T.; Zhao, W. X.; Nie, J.-Y.; and Wen, J.-R. 2024. Pre-trained language models for text generation: A survey. *ACM Computing Surveys*, 56(9): 1–39.
- Masterman, T.; Besen, S.; Sawtell, M.; and Chao, A. 2024. The landscape of emerging ai agent architectures for reasoning, planning, and tool calling: A survey. *arXiv preprint arXiv:2404.11584*.
- Nelson, E.; Kollias, G.; Das, P.; Chaudhury, S.; and Dan, S. 2024. Needle in the Haystack for Memory Based Large Language Models. *arXiv preprint arXiv:2407.01437*.
- Packer, C.; Wooders, S.; Lin, K.; Fang, V.; Patil, S. G.; Stoica, I.; and Gonzalez, J. E. 2023. MemGPT: Towards LLMs as Operating Systems. *arXiv preprint arXiv:2310.08560*.
- Park, J. S.; O’Brien, J.; Cai, C. J.; Morris, M. R.; Liang, P.; and Bernstein, M. S. 2023. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th annual acm symposium on user interface software and technology*, 1–22.
- Shinn, N.; Cassano, F.; Berman, E.; Gopinath, A.; Narasimhan, K.; and Yao, S. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. *arXiv preprint arXiv:2303.11366*.
- Tang, Q.; Xiang, H.; Yu, L.; Yu, B.; Lu, Y.; Han, X.; Sun, L.; Zhang, W.; Wang, P.; Liu, S.; et al. 2025. Beyond turn limits: Training deep search agents with dynamic context window. *arXiv preprint arXiv:2510.08276*.
- Wang, Y.; Wang, Z.; Shi, Y.; Chen, S.; Wang, X.; Wang, Y.; Shen, B.; Li, L.; Gu, X.; McAuley, J.; and Zeng, D. D. 2026. Context Compression for LLM Agents: A Survey of Methods, Failure Modes, and Evaluation. *Preprints*.
- Yao, S.; Zhao, J.; Yu, D.; Du, N.; Shafran, I.; Narasimhan, K.; and Cao, Y. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations*.

Supplementary

Formal Guarantees for Addressable Recall Compaction

This appendix formalizes three statements: (i) why keeping every citation simultaneously visible is incompatible with a turn-independent prompt bound, (ii) why a paginated ARC implementation provides a bounded active view and exact address-conditioned recall, and (iii) why linear external-memory growth is unavoidable for worst-case exact recovery.

Formal Model

Definition 2 (Token sequences and observation history). Let Σ be a finite token alphabet, and let Σ^* denote the set of finite token sequences. For $x \in \Sigma^*$, let $|x|$ denote its token length. At turn i , let $\mathcal{O}_i = (O_1, O_2, \dots, O_{N_i})$ be the ordered sequence of tool observations that have been produced by normal environment actions up to that turn. For each observation O_j , define $n_j = |O_j|$. The theorem below concerns exact preservation of the observations O_j . It does not claim that discarded hidden reasoning traces are recoverable unless those traces are also stored verbatim.

Definition 3 (Usable prompt budget). Let L_{ctx} be the model’s total context limit and let G_{max} be the maximum number of completion tokens reserved for one model call. Define the usable input-token budget by

$$L = L_{\text{ctx}} - G_{\text{max}}. \quad (4)$$

Thus, any model invocation whose serialized input contains at most $\bar{L}$ tokens remains within the total context limit whenever the generated completion contains at most $G_{\max}$ tokens.

Assumption 4 (Injective fixed-width occurrence identifiers). Fix a declared maximum number of stored observation occurrences $N_{\max} \geq 2$, and define

$$b = \lceil \log_2 (N_{\max} + 1) \rceil. \quad (5)$$

For each observation occurrence $j \in \{1, \dots, N_{\max}\}$, let $\text{enc}_b(j)$ be its injective b -bit encoding. Let fp_d be any deterministic d -bit fingerprint function. ARC assigns

$$\text{id}_j = \text{enc}_b(j) \parallel \text{fp}_d(\text{signature}(A_j) \parallel 0 \times 1\text{F} \parallel O_j). \quad (6)$$

The occurrence index, rather than the fingerprint, provides uniqueness. A SHA1 value may be retained as the fingerprint, but none of the proofs assumes collision resistance of SHA1 or of a truncated digest.

Assumption 5 (Append-only observation store). Immediately after a normal action A_j produces observation O_j , and before any compaction can remove O_j from the active transcript, ARC writes a record under id_j satisfying

$$\text{ObsStore}[\text{id}_j].\text{full_content} = O_j. \quad (7)$$

After this write, the record is never overwritten, truncated, or deleted during the task.

Assumption 6 (Persistent external catalog with bounded pages). For every stored observation occurrence j , ARC appends a citation stub z_j to an external catalog. The stub contains id_j , bounded metadata, bounded previews, and a recall hint. Its serialized token charge, including delimiters, is at most a fixed constant $c \geq 1$. Fix a page capacity $p \geq 1$. At turn i , page s is

$$\mathcal{G}_{i,s} = z_{(s-1)p+1} \parallel \dots \parallel z_{\min\{sp, N_i\}}, \quad (8)$$

for $1 \leq s \leq \lceil N_i/p \rceil$. Only one page is materialized in the active prompt at a time. Define the catalog-page token budget P so that $pc \leq P$. Catalog navigation is stateful, so commands such as `catalog-first` and `catalog-next` have bounded prompt cost.

Assumption 7 (Budgeted active view). Let $F = [\text{system}, g]$ be the fixed task prefix, and define $B = \text{tokens}(F)$. Before every call to Model, ARC constructs the active transcript

$$\text{transcript}'_i = \text{Serialize}(F, S_i, H_i, G_i, U_i), \quad (9)$$

where S_i is the deterministic summary, H_i is the recent suffix, G_i is one catalog page, and U_i is the materialized recall buffer. Their budgets satisfy

$$\text{tokens}(S_i) \leq M, \quad (10)$$

$$\text{tokens}(H_i) \leq R, \quad (11)$$

$$\text{tokens}(G_i) \leq P, \quad (12)$$

$$\text{tokens}(U_i) \leq Q. \quad (13)$$

The recent suffix is selected by a token budget R , not by retaining a fixed number of turn pairs. Let η_i be the exact serialization overhead for role markers, separators, field names, and control instructions at turn i . Assume $0 \leq \eta_i \leq \eta$ for a

fixed constant η , and assume that the serializer is charged so that

$$\text{tokens}(\text{transcript}'_i) \leq B + \text{tokens}(S_i) + \text{tokens}(H_i) + \text{tokens}(G_i) + \text{tokens}(U_i) + \eta_i. \quad (14)$$

Define

$$K = B + M + R + P + Q + \eta, \quad (15)$$

and require $K \leq L$. If a newly produced observation is too large to fit in H_i , ARC stores it first and represents it by its bounded citation stub before the next model invocation.

Assumption 8 (Exact chunked recall). Fix a positive recall payload budget $q \geq 1$ and a bounded response-wrapper cost $\rho \geq 0$ satisfying $q + \rho \leq Q$. For every nonempty stored observation O_j , define the number of recall chunks by $m_j = \lceil n_j/q \rceil$. For each chunk index $\ell \in \{1, \dots, m_j\}$, define

$$a_{j,\ell} = (\ell - 1)q + 1, \quad (16)$$

$$b_{j,\ell} = \min\{\ell q, n_j\}, \quad (17)$$

and define the returned payload by

$$\mathcal{R}_{j,\ell} = O_j[a_{j,\ell} : b_{j,\ell}]. \quad (18)$$

The commands `_recall-start id` and `_recall-next id` return these chunks in increasing order of ℓ using an external cursor. A recall reads only `ObsStore`; it never invokes `Execute`. The recall buffer applies eviction if necessary so that its total serialized token cost remains at most Q . For an empty observation, define $m_j = 0$, and let the empty concatenation recover the empty sequence.

Why Simultaneously Visible Citation Persistence Cannot Be Bounded

Proposition 9 (Impossibility of all-visible citation persistence). *Suppose that, after citing n distinct observations, all n citation stubs must remain simultaneously visible in the active prompt. If each visible citation incurs a token cost of at least $c_{\min} > 0$, then no constant K can bound the active prompt length for arbitrarily large n .*

Proof. Because all n citation stubs are simultaneously visible and each requires at least $c_{\min}$ tokens, the active prompt length $C(n)$ satisfies $C(n) \geq nc_{\min}$.

Suppose, for contradiction, that there exists a constant K such that $C(n) \leq K$ for every positive integer n . Choose

$$n^* = \left\lfloor \frac{K}{c_{\min}} \right\rfloor + 1. \quad (19)$$

By the defining property of the floor function,

$$\left\lfloor \frac{K}{c_{\min}} \right\rfloor \leq \frac{K}{c_{\min}} < \lfloor K/c_{\min} \rfloor + 1 = n^*. \quad (20)$$

Thus, $K/c_{\min} < n^*$. Since $c_{\min} > 0$, multiplying both sides by $c_{\min}$ yields $K < n^*c_{\min}$. Consequently,

$$C(n^*) \geq n^*c_{\min} > K, \quad (21)$$

contradicting the assumed bound $C(n^*) \leq K$.

Therefore, a turn-independent bound on the active prompt is incompatible with keeping all citation stubs simultaneously visible. Persistent citation addressability must instead be maintained in external state, with only a bounded page or subset materialized in the active prompt at any given time. $\square$

Supporting Lemmas

Lemma 10 (Identifier uniqueness). *Under Assumption 4, the identifiers $\text{id} * 1, \dots, \text{id} * N_{\max}$ are pairwise distinct.*

Proof. Fix any distinct indices $j, k \in 1, \dots, N_{\max}$. Suppose, for contradiction, that $\text{id}_j = \text{id}_k$. Each identifier is formed by concatenating a b -bit occurrence encoding with a d -bit fingerprint. Because both components have fixed width, equality of the complete identifiers implies equality of their first b bits:

$$\text{enc}_b(j) = \text{enc}_b(k). \quad (22)$$

By Assumption 4, $\text{enc} * b$ is injective on $1, \dots, N * \max$, so this equality implies $j = k$, contradicting the choice of distinct indices. Therefore, $\text{id}_j \neq \text{id}_k$. Since j and k were arbitrary, the identifiers $\text{id} * 1, \dots, \text{id} * N * \max$ are pairwise distinct. $\square$

Lemma 11 (Uniform active-view bound). *Under Assumption 7, every model invocation satisfies*

$$\text{tokens}(\text{transcript}'_i) \leq K \leq L. \quad (23)$$

The bound does not depend on i or on N_i .

Proof. By the serializer bound in Assumption 7,

$$\text{tokens}(\text{transcript}'_i) \leq B + \text{tokens}(S_i) + \text{tokens}(H_i) + \text{tokens}(G_i) + \text{tokens}(U_i) + \eta_i. \quad (24)$$

Applying the component-wise budget inequalities gives

$$\text{tokens}(\text{transcript}'_i) \leq B + M + R + P + Q + \eta. \quad (25)$$

By the definition of K ,

$$B + M + R + P + Q + \eta = K. \quad (26)$$

Therefore,

$$\text{tokens}(\text{transcript}'_i) \leq K. \quad (27)$$

The required budget condition gives $K \leq L$. Combining the last two inequalities yields

$$\text{tokens}(\text{transcript}'_i) \leq K \leq L. \quad (28)$$

The quantities B, M, R, P, Q , and η are fixed for the task and do not contain i or N_i . Hence K is independent of both i and N_i . $\square$

Lemma 12 (Bounded-page persistence and finite discoverability). *Under Assumption 6, every citation stub z_j , for $1 \leq j \leq N_i$, appears on exactly one catalog page, and each page has token cost at most P . Starting from the first page and repeatedly issuing `catalog-next`, the page containing z_j is reached within $\lceil j/p \rceil$ page materializations and, consequently, within $\lceil N_i/p \rceil$ page materializations.*

Proof. Fix $j \in 1, \dots, N_i$, and define $s_j = \lceil j/p \rceil$. By the defining property of the ceiling function,

$$s_j - 1 < \frac{j}{p} \leq s_j. \quad (29)$$

Since $p > 0$, multiplying throughout by p yields

$$(s_j - 1)p < j \leq s_j p. \quad (30)$$

Because j is an integer, the strict lower bound implies $(s_j - 1)p + 1 \leq j$. Hence,

$$(s_j - 1)p + 1 \leq j \leq s_j p. \quad (31)$$

By the page definition in Assumption 6, z_j therefore appears on page s_j .

To establish uniqueness, suppose, for contradiction, that z_j also appears on a distinct page r . Without loss of generality, let $r > s_j$. Membership in page s_j implies $j \leq s_j p$, whereas membership in page r implies $j \geq (r - 1)p + 1$. Since $r \geq s_j + 1$, we have $(r - 1)p + 1 \geq s_j p + 1$, and therefore $j \geq s_j p + 1$. This contradicts $j \leq s_j p$. Thus, z_j appears on exactly one catalog page.

Each page contains at most p citation stubs, and each stub incurs a token cost of at most c . The token cost of a page is therefore at most pc . By Assumption 6, $pc \leq P$, so every materialized catalog page has token cost at most P .

Finally, a sequential scan beginning at the first page reaches page s_j after $s_j = \lceil j/p \rceil$ page materializations. Since $j \leq N_i$ and the ceiling function is monotone,

$$\left\lceil \frac{j}{p} \right\rceil \leq \left\lceil \frac{N_i}{p} \right\rceil. \quad (32)$$

Therefore, the page containing z_j is discoverable within $\lceil N_i/p \rceil$ page materializations. $\square$

Lemma 13 (Exact non-overlapping chunk reconstruction). *Under Assumption 8, for every nonempty observation O_j , the chunks $\mathcal{R}_{j,1}, \dots, \mathcal{R}_{j,m_j}$ form a disjoint, order-preserving partition of O_j . In particular,*

$$\mathcal{R}_{j,1} \parallel \mathcal{R}_{j,2} \parallel \dots \parallel \mathcal{R}_{j,m_j} = O_j. \quad (33)$$

Moreover, every chunk has at most q payload tokens, and the number of chunks is exactly $m_j = \lceil n_j/q \rceil$.

Proof. Fix a nonempty observation O_j , and write $n = n_j$ and $m = m_j = \lceil n/q \rceil$. Since $n \geq 1$ and $q \geq 1$, we have $m \geq 1$. By the defining property of the ceiling function,

$$m - 1 < \frac{n}{q} \leq m, \quad (34)$$

and hence

$$(m - 1)q < n \leq mq. \quad (35)$$

We first show that the chunks cover every token position. Fix any $t \in 1, \dots, n$ and define $\ell(t) = \lceil t/q \rceil$. Because $1 \leq t \leq n$ and the ceiling function is monotone,

$$1 \leq \ell(t) \leq \left\lceil \frac{n}{q} \right\rceil = m. \quad (36)$$

Moreover, the definition of $\ell(t)$ gives

$$(\ell(t) - 1)q < t \leq \ell(t)q. \quad (37)$$

Since t is an integer, the strict left inequality implies $(\ell(t) - 1)q + 1 \leq t$. Therefore,

$$a_{j,\ell(t)} = (\ell(t) - 1)q + 1 \leq t. \quad (38)$$

Because $t \leq \ell(t)q$ and $t \leq n$, we also have

$$t \leq \min \ell(t)q, n = b_{j,\ell(t)}. \quad (39)$$

Thus $a_{j,\ell(t)} \leq t \leq b_{j,\ell(t)}$, so every token position belongs to at least one chunk.

We next establish that the chunks are pairwise disjoint. Suppose, for contradiction, that some token position t belongs to two chunks indexed by $r < s$. Membership in chunk r implies

$$t \leq b_{j,r} = \min rq, n \leq rq. \quad (40)$$

Membership in chunk s implies

$$t \geq a_{j,s} = (s - 1)q + 1. \quad (41)$$

Since $s \geq r + 1$, it follows that $(s - 1)q + 1 \geq rq + 1$, and therefore $t \geq rq + 1$. This contradicts $t \leq rq$. Hence, no token position belongs to more than one chunk.

It remains to verify the chunk lengths. For every $\ell \in 1, \dots, m - 1$, the inequality $(m - 1)q < n$ implies $\ell q < n$, and hence $b_{j,\ell} = \ell q$. Therefore,

$$|\mathcal{R} * j, \ell| = b * j, \ell - a_{j,\ell} + 1 = \ell q - ((\ell - 1)q + 1) + 1 = q. \quad (42)$$

For the final chunk, $n \leq mq$ gives $b_{j,m} = n$, so

$$|\mathcal{R} * j, m| = n - ((m - 1)q + 1) + 1 = n - (m - 1)q. \quad (43)$$

The inequalities $(m - 1)q < n \leq mq$ further imply

$$1 \leq |\mathcal{R} * j, m| \leq q. \quad (44)$$

Thus every chunk contains at most q payload tokens, and the total payload length is

$$\begin{aligned} \sum_{\ell=1}^m |\mathcal{R} * j, \ell| &= \sum * \ell = 1^{m-1}q + (n - (m - 1)q) \\ &= (m - 1)q + n - (m - 1)q \\ &= n. \end{aligned} \quad (45)$$

The chunk intervals are disjoint, exhaustive, and ordered by ℓ . Their concatenation therefore recovers all n tokens of O_j in their original order:

$$\mathcal{R} * j, 1 \parallel \dots \parallel \mathcal{R} * j, m = O_j. \quad (46)$$

Finally, the number of chunks is $m = \lceil n/q \rceil$ by definition, completing the proof. $\square$

Corollary 14 (Recall-action and serialization overhead). *For every nonempty observation O_j , the number of exact recall responses satisfies*

$$\frac{n_j}{q} \leq m_j < \frac{n_j}{q} + 1. \quad (47)$$

The total recalled payload is exactly n_j tokens. If each response has wrapper cost at most ρ , the total serialized size of all recall responses is at most

$$n_j + \rho \left(\frac{n_j}{q} + 1 \right). \quad (48)$$

Proof. By definition, $m_j = \lceil n_j/q \rceil$. For any real number x , the ceiling function satisfies $x \leq \lceil x \rceil < x + 1$. Substituting $x = n_j/q$ therefore gives $n_j/q \leq m_j < n_j/q + 1$. By Lemma 13, the recalled responses collectively contain exactly the original payload:

$$\sum_{\ell=1}^{m_j} |\mathcal{R}_{j,\ell}| = n_j. \quad (49)$$

Because each of the m_j responses contributes at most ρ wrapper tokens, their total serialized size is at most $n_j + \rho m_j$. Moreover, since $\rho \geq 0$ and $m_j < n_j/q + 1$, multiplying by ρ yields

$$\rho m_j \leq \rho (n_j/q + 1). \quad (50)$$

Consequently,

$$n_j + \rho m_j \leq n_j + \rho (n_j/q + 1), \quad (51)$$

which establishes the claimed upper bound. $\square$

Main Guarantee

Theorem 15 (Bounded active view, observational losslessness, and exact address-conditioned recall). *Consider any ARC execution satisfying Assumptions 4–8, and suppose $N_i \leq N_{\max}$ at turn i . Then all of the following statements hold.*

1. **Prompt safety.** *Every model invocation uses an active transcript satisfying*

$$\text{tokens}(\text{transcript}'_i) \leq K \leq L. \quad (52)$$

2. **Stable archival fidelity.** *For every $j \in \{1, \dots, N_i\}$ and every later turn $t \geq i$ before the declared capacity is exhausted, the identifier id_j remains unique and*

$$\text{ObsStore}_t[\text{id}_j].\text{full_content} = O_j. \quad (53)$$

3. **Persistent finite discoverability.** *The citation stub for every O_j remains in the external catalog and can be materialized by a finite page scan. At most one page of token cost P is visible at a time.*
4. **Exact address-conditioned recall without re-execution.** *Given a valid id_j , ARC returns O_j exactly in*

$$m_j = \left\lceil \frac{n_j}{q} \right\rceil \quad (54)$$

non-overlapping recall responses, each containing at most q payload tokens, and no recall response invokes $\text{Execute}(A_j)$.

5. **Observational losslessness.** *The ordered observation history $\mathcal{O}_i$ can be reconstructed exactly from ObsStore and the stored creation indices. Consequently, the map from ordered observation histories to the archived system state is injective.*
6. **Separation of active and archival growth.** *The active-view bound K is independent of i and N_i , whereas the exact archived payload equals $\sum_{j=1}^{N_i} n_j$ tokens, excluding metadata.*

Proof. We prove the six claims in order.

1. Prompt safety. Lemma 11 gives

$$\text{tokens}(\text{transcript}'_i) \leq K \leq L. \quad (55)$$

This holds before every model invocation by Assumption 7. Because $L = L_{\text{ctx}} - G_{\text{max}}$, adding any completion of at most G_{max} tokens gives

$$\text{tokens}(\text{transcript}'_i) + G_{\text{max}} \leq L + G_{\text{max}}. \quad (56)$$

Substituting the definition of L gives

$$L + G_{\text{max}} = (L_{\text{ctx}} - G_{\text{max}}) + G_{\text{max}}. \quad (57)$$

Cancelling $-G_{\text{max}}$ and $+G_{\text{max}}$ yields

$$L + G_{\text{max}} = L_{\text{ctx}}. \quad (58)$$

Therefore,

$$\text{tokens}(\text{transcript}'_i) + G_{\text{max}} \leq L_{\text{ctx}}. \quad (59)$$

Thus the prompt and reserved completion fit within the total context limit.

2. Stable archival fidelity. Identifier uniqueness follows from Lemma 10. We now prove persistence of the stored content by induction over host-system transitions after the insertion of O_j .

Immediately after insertion, Assumption 5 gives

$$\text{ObsStore}[\text{id}_j].\text{full_content} = O_j. \quad (60)$$

This is the induction base case. Assume that after some later host-system transition u , the equality still holds:

$$\text{ObsStore}_u[\text{id}_j].\text{full_content} = O_j. \quad (61)$$

Consider transition $u + 1$. There are three relevant cases.

First, a new normal action may insert a new observation O_k . Because it is a new occurrence, $k \neq j$. Lemma 10 gives $\text{id}_k \neq \text{id}_j$. The insertion under id_k therefore does not overwrite the record under id_j .

Second, a recall action reads an existing record. By Assumption 8, recall performs no write to ObsStore . Hence the record under id_j is unchanged.

Third, a compaction or catalog-navigation action modifies only the materialized active view, summary, recent suffix, page selection, or recall buffer. By Assumption 5, none of these operations overwrites, truncates, or deletes the record under id_j .

In every case,

$$\text{ObsStore}_{u+1}[\text{id}_j].\text{full_content} = O_j. \quad (62)$$

The induction step is complete. Therefore the equality holds at every later turn t .

3. Persistent finite discoverability. Assumption 6 appends z_j to an external catalog and never removes it. Lemma 12 shows that z_j belongs to exactly one page, that the page costs at most P tokens, and that a sequential page scan reaches it after at most

$$\left\lceil \frac{j}{p} \right\rceil \leq \left\lceil \frac{N_i}{p} \right\rceil \quad (63)$$

page materializations. Thus the citation remains persistently discoverable without requiring all citations to be visible simultaneously.

4. Exact address-conditioned recall without re-execution. Fix any $j \in \{1, \dots, N_i\}$. By Claim 2, the store entry remains exactly equal to O_j . Lemma 13 gives

$$\mathcal{R}_{j,1} \parallel \dots \parallel \mathcal{R}_{j,m_j} = O_j, \quad (64)$$

and each chunk has at most q payload tokens. Assumption 8 states that the recall handler reads these chunks from ObsStore and never invokes Execute . Therefore the original environment action is not re-executed, and changes in the external environment after the original action cannot alter the archived return value.

5. Observational losslessness. Define a decoder D_i that sorts the archive records by their stored creation indices and outputs their `full_content` fields:

$$D_i(\text{ObsStore}_i) = \left(\text{ObsStore}_i[\text{id}_1].\text{full_content}, \dots, \text{ObsStore}_i[\text{id}_{N_i}].\text{full_content} \right). \quad (65)$$

By Claim 2, each component satisfies

$$\text{ObsStore}_i[\text{id}_j].\text{full_content} = O_j. \quad (66)$$

Substituting these equalities component by component gives

$$D_i(\text{ObsStore}_i) = (O_1, O_2, \dots, O_{N_i}). \quad (67)$$

By Definition 2,

$$(O_1, O_2, \dots, O_{N_i}) = \mathcal{O}_i. \quad (68)$$

Hence

$$D_i(\text{ObsStore}_i) = \mathcal{O}_i. \quad (69)$$

To prove injectivity, suppose two ordered observation histories $\mathcal{O}_i$ and $\tilde{\mathcal{O}}_i$ produce the same archived system state. Applying the same deterministic decoder to that state gives

$$D_i(\text{ObsStore}_i) = \mathcal{O}_i \quad (70)$$

and

$$D_i(\text{ObsStore}_i) = \tilde{\mathcal{O}}_i. \quad (71)$$

By transitivity of equality,

$$\mathcal{O}_i = \tilde{\mathcal{O}}_i. \quad (72)$$

Therefore distinct ordered observation histories cannot map to the same archived state, so the map is injective.

6. Separation of active and archival growth. By Claim 1, the active prompt is bounded by

$$K = B + M + R + P + Q + \eta, \quad (73)$$

where every term on the right-hand side is fixed independently of i and N_i . Therefore the active-view size is uniformly bounded.

By Assumption 5, the archive stores one verbatim payload O_j for each occurrence j . The payload token count is therefore

$$\begin{aligned} \text{PayloadTokens}(\text{ObsStore}_i) &= |O_1| + |O_2| + \dots + |O_{N_i}| \\ &= n_1 + n_2 + \dots + n_{N_i} \\ &= \sum_{j=1}^{N_i} n_j. \end{aligned} \tag{74}$$

This quantity may grow with the trajectory, but it grows only in external archival state, not in the active prompt. This proves the claimed separation. $\square$

Remark 16 (Scope of the guarantee). Theorem 15 is a mechanism-level guarantee. It proves prompt safety, exact archival preservation, finite discoverability, and exact responses to valid recall commands. It does not prove that the language model will select the correct identifier, issue enough recall commands, or solve the downstream task correctly. It is also an *observation-losslessness* theorem. If the paper wishes to claim full-transcript losslessness, then every discarded action and every discarded reasoning record must also be archived under the same append-only discipline.

Worst-Case External-Memory Lower Bound

Proposition 17 (Exact recovery requires linearly growing external memory). *Consider any deterministic memory system that receives an arbitrary N -bit history $x \in \{0, 1\}^N$. Suppose the active representation has at most 2^κ distinguishable states, the external memory has at most 2^s distinguishable states, and a deterministic decoder must recover every x exactly from the pair of states. Then*

$$s \geq N - \kappa. \tag{75}$$

Consequently, for fixed κ , worst-case exact recovery requires external storage in $\Omega(N)$ bits. A prompt backed by κ fixed binary storage cells is a special case because it has exactly 2^κ possible cell configurations.

Proof. There are exactly 2^N possible histories in $0, 1^N$. By assumption, the active representation and external memory can occupy at most 2^κ and 2^s distinguishable states, respectively. Hence, the total number of distinguishable active-external state pairs is at most $2^\kappa 2^s = 2^{\kappa+s}$. Exact recovery of every history requires the encoding from histories to state pairs to be injective: if two distinct histories were mapped to the same state pair, a deterministic decoder would receive identical inputs for both and therefore could not recover both correctly. Consequently, $2^N \leq 2^{\kappa+s}$. Since 2^x is strictly increasing, taking base-two logarithms yields $N \leq \kappa + s$, or equivalently, $s \geq N - \kappa$. For fixed κ and any $N \geq 2\kappa$, we have $\kappa \leq N/2$, and thus $N - \kappa \geq N/2$. Therefore, $s \geq N - \kappa \geq N/2$ for all $N \geq 2\kappa$, establishing that $s \in \Omega(N)$. $\square$

Corollary 18 (First-order payload optimality of append-only ARC). *For an incompressible N -bit observation history and a fixed active-state budget κ , Proposition 17 establishes a*

worst-case lower bound of $N - \kappa$ bits of external storage. An append-only ARC archive stores exactly N observation-payload bits, excluding metadata, and therefore exceeds this lower bound by only $N - (N - \kappa) = \kappa$ bits. Moreover, for $N > \kappa$, $(N/(N - \kappa)) = (1/(1 - \kappa/N))$, which implies that $\lim_{N \rightarrow \infty} (N/(N - \kappa)) = 1$. Hence, among methods that support exact recovery of arbitrary histories while maintaining a bounded active state, ARC’s verbatim payload storage is asymptotically optimal to first order.

Proof. The lower bound of $N - \kappa$ bits follows directly from Proposition 17. Because ARC stores all N input bits verbatim, its payload storage is exactly N bits. Therefore, its additive overhead relative to the lower bound is $N - (N - \kappa) = \kappa$ bits.

To analyze the multiplicative overhead, divide both the numerator and denominator by $N > 0$:

$$\frac{N}{N - \kappa} = \frac{1}{1 - \kappa/N}. \tag{76}$$

For fixed κ ,

$$\lim_{N \rightarrow \infty} \frac{\kappa}{N} = 0, \tag{77}$$

and hence

$$\lim_{N \rightarrow \infty} \left(1 - \frac{\kappa}{N}\right) = 1. \tag{78}$$

By continuity of the reciprocal function at 1, it follows that

$$\lim_{N \rightarrow \infty} \frac{1}{1 - \kappa/N} = 1. \tag{79}$$

Thus, the ratio between ARC’s payload storage and the information-theoretic lower bound converges to 1, establishing ARC’s first-order asymptotic optimality. $\square$

Remark 19 (What the lower bound does and does not say). The lower bound is a worst-case result that applies to the exact recovery of arbitrary histories. Compressible histories may require less external storage, and the bound excludes implementation-specific metadata. Its purpose is to establish that a bounded active prompt and exact recoverability cannot both be achieved with constant total memory: any information removed from the prompt must be represented elsewhere in the external memory.

Ablation

ARC’s behavior is governed by five parameters, each of which trades off how often the compaction fires against how much of `ObsStore` is allowed back into the visible transcript at once. We describe them here as part of the method because each is varied independently in our Ablation Section; together they bound the conditions under which ARC’s guarantee is exercised versus dormant.

Context-window budget (*max_model_len*, *L*).

Default=16,384. The token limit the serving engine enforces on prompt + completion length (Section 2.1); this is exactly *L* in Algorithm 1. Smaller values make the overflow condition $C_i > L$ true more often, so compaction — and with it ARC’s citation-and-recall mechanism — is exercised more frequently; larger values

let the same trajectory run further before fires, and at the limit ($L \rightarrow \infty$) ARC, Auto-Compact, and full-context converge because none of them are ever invoked.

Recall working-set budget (*recall_budget_chars*).

Default=16,000. The total character budget for observations currently *materialized* — i.e., fetched back via `_recall` and expanded verbatim in the transcript — at once. This is a constraint on the recall path rather than on `ObsStore` itself: `ObsStore` is never bounded, but the *live, expanded* content drawn from it is capped so that recall cannot itself re-create the unbounded growth ARC is designed to avoid. When a new `_recall $id` would push the materialized total over this budget, the least-recently-used materialized observation is evicted back to its citation stub (its `$id` remains visible; only the expansion is reverted).

Per-recall cap (*recall_max_chars*). Default=8,000. The maximum size of a single `_recall $id` response. If the stored observation is larger than this, the response is windowed to a head/tail excerpt rather than injected in full — the same head+tail structure used in the citation stub, just with a larger window. This bounds the worst case where one oversized observation (e.g., a full-repository `grep` dump) would otherwise consume the entire recall working-set budget in a single call.

Forced-compaction schedule (*force_at_calls*).

Default=(5,10,15). A set of call indices at which compaction is triggered experimentally regardless of whether C_i has organically exceeded L . This lets the compaction *frequency* be controlled directly, isolating its effect from the task- and model-specific rate at which context happens to grow, and is used only in the evaluation, not in deployment, where compaction fires solely on a overflow.

Fixture size (*noise_lines_per_side*). Default=25 as a benchmark parameter. Not a parameter of ARC itself but of the needle-in-a-haystack harness: the number of distractor lines placed on each side of the needle banner inside input file, which determines the size of the observation that compaction must decide whether to keep, drop, or cite (at the default, $\approx 4.5k$ characters — above the 500-character keep-verbatim threshold used by all lossy baselines, but below the 8,000-character limit at which the *presentation* layer would elide the needle before compaction ever runs). Increasing it raises the cost of losing the needle under lossy compaction while leaving ARC’s citation size unaffected, since a citation’s cost is bounded by the head/tail preview rather than by the underlying observation’s length.

Recall_budget for Longbench task

Setup. We ablate the recall budget by jointly sweeping `RECALL_BUDGET_CHARS` and `RECALL_MAX_CHARS` (Table 8) at three levels—8k/4k, 16k/8k, and 32k/16k—across two model scales (Qwen3-8B, 16k context; Qwen3-32B, 32k context) and five memory/compaction methods (`FULL_CONTEXT`, `SLIDING_WINDOW`, `LLM_SUMMARY`, `STRUCTURED_STATE`, `RAG_MEMORY`) compared against our

method, ARC. Accuracy is computed as

$$\text{Accuracy} = \frac{\text{Success}}{\text{Success} + \text{Fail}}$$

over the LongBench evaluation set ($n = 311$ hard tasks).

Full_context collapses at every setting. `FULL_CONTEXT` scores 0.00% accuracy across all budgets and both scales, confirming that LongBench inputs exceed the base context window regardless of the recall allowance and establishing that some form of compaction is required rather than optional.

ARC’s accuracy is a single-peaked, inverted-U function of recall budget, not monotonic. On Qwen3-8B, ARC accuracy moves from 26.05% $\rightarrow$ 29.26% $\rightarrow$ 27.33% as the budget grows from 8k/4k to 16k/8k to 32k/16k; on Qwen3-32B it moves from 29.90% $\rightarrow$ 34.08% $\rightarrow$ 33.76%. In both cases the peak occurs at the intermediate 16k/8k budget rather than the largest. This indicates ARC’s recall mechanism benefits from a moderate, well-targeted budget and degrades once the budget grows large enough to admit lower-relevance content, providing evidence that ARC’s gains come from *recall quality* rather than simply retrieving more.

ARC is the best or near-best method at practical (16k/8k and 32k/16k) budgets on both scales.

Scale	Budget	ARC	Runner-up	Margin
8B	16/8k	29.26%	Structured_state	26.05% +3.2pp
8B	32/16k	27.33%	RAG_memory	24.76% +2.6pp
32B	16/8k	34.08%	Sliding_window	30.87% +3.2pp
32B	32/16k	33.76%	Sliding_window	29.58% +4.2pp

At the smallest budget (8k/4k), ARC is competitive but not the top performer: `SLIDING_WINDOW` leads on Qwen3-32B (31.19%), while `LLM_SUMMARY` and `STRUCTURED_STATE` slightly outperform ARC on Qwen3-8B. This suggests ARC requires a minimum recall budget to construct a useful memory structure, which we note as a limitation under very tight budgets rather than omit.

Baseline methods do not show the same peak-then-decline profile as ARC, and some degrade sharply. `LLM_SUMMARY` is flat-to-declining as budget grows on Qwen3-32B (24.76% $\rightarrow$ 22.51% $\rightarrow$ 24.12%) and peaks early before dropping on Qwen3-8B (26.69% $\rightarrow$ 26.69% $\rightarrow$ 23.15%). `Structured_state` exhibits the steepest degradation of any method on Qwen3-8B, falling from 27.01% to 21.22% (−5.8pp) as the budget increases from 8k/4k to 32k/16k, consistent with excess retrieved state introducing noise rather than signal. `SLIDING_WINDOW` and `RAG_MEMORY` are comparatively flat or monotonically decreasing with budget; they neither benefit from additional budget in the transient manner observed for ARC nor collapse as sharply as `STRUCTURED_STATE`.

ARC’s advantage grows with model scale. The margin between ARC and the next-best method is larger on Qwen3-32B than on Qwen3-8B at matched budgets (e.g., +4.2pp vs. +2.6pp at 32k/16k), suggesting ARC’s recall mechanism scales favorably with model capacity, likely because larger

models make better use of the structured, compacted recall content provided by ARC.

Recall_budget for Needle-in-a-haystack task

For saving the computational cost, we only ablate recall_budget for the Needle-in-a-haystack on small model Qwen3-8B in Table 7.

Setup. We ablate the recall budget by jointly sweeping ‘RECALL_BUDGET_CHARS’ and ‘RECALL_MAX_CHARS’ at four levels — 4k/2k, 8k/4k, 16k/8k, and 32k/16k — on a synthetic needle-in-a-haystack retrieval task ($n = 1000$), comparing FULL_CONTEXT, SLIDING_WINDOW, LLM_SUMMARY, STRUCTURED_STATE, and RAG_MEMORY against ARC. Accuracy is Success / (Success + Fail).

ARC scales monotonically with recall budget and saturates near ceiling, unlike on LongBench. ARC accuracy rises 92.5% $\rightarrow$ 93.2% $\rightarrow$ 99.1% $\rightarrow$ 99.4% as budget grows from 4k/2k to 32k/16k, with the large jump occurring at 16k/8k and only +0.3pp gained from doubling the budget again at 32k/16k. This is the opposite budget-sensitivity profile from the LongBench ablation (where ARC peaked at an intermediate budget and declined thereafter): on this synthetic, single-needle retrieval task there is little irrelevant content to be diluted by a larger budget, so additional recall capacity is purely additive until the task is essentially solved ($\sim 99\%$ accuracy, near the practical ceiling given task noise).

ARC’s margin over the next-best baseline widens as budget increases, then holds. At the smallest budget (4k/2k) ARC leads the next-best method (RAG_MEMORY, 80.0%) by +12.5pp; at 8k/4k it leads LLM_SUMMARY (78.4%) by +14.8pp; at 16k/8k and 32k/16k it leads RAG_MEMORY (78.8% and 79.3%) by +20.3pp and +20.1pp respectively. The margin roughly doubles once budget exceeds 8k/4k, indicating ARC converts additional recall budget into accuracy far more effectively than any baseline.

Baseline methods plateau well below ARC’s ceiling regardless of budget, showing their limitation is retrieval quality, not budget size. Full_context (35–40%) and Sliding_window (53–57%) are essentially flat across the entire budget sweep — a 4x increase in recall budget from 4k/2k to 16k/8k changes their accuracy by only a few points, meaning these methods are bottlenecked by their retrieval mechanism rather than by capacity. LLM_SUMMARY, STRUCTURED_STATE, AND RAG_MEMORY occupy a middle band (69–80%) that is likewise budget-insensitive: RAG_MEMORY, the strongest baseline, ranges only 77.5–80.0% across the full sweep and never approaches ARC’s 99%+ accuracy at matched budgets.

Contrast with the LongBench ablation strengthens the paper’s core claim. Together, the two ablations show that ARC’s recall-budget behavior is task-dependent in a way baselines’ is not: on LongBench (dense, multi-fact, real-world text) ARC shows an inverted-U with an optimum at moderate budget, while on needle-in-a-haystack (sparse, single-fact, synthetic text) ARC shows monotonic improvement to near-ceiling. Baselines, by contrast, are budget-insensitive on both tasks — they neither peak nor saturate,

they simply plateau below their respective ceilings. This supports the claim that ARC’s improvements stem from *how* it allocates and prioritizes recall content, not merely from having more of it.

Method	Recall Budget	Success	Fail	Accuracy (%)
Full context	4k/2k	375	625	37.5
Sliding window		570	430	57.0
LLM summary		762	238	76.2
Structured state		730	270	73.0
RAG memory		800	200	80.0
ARC		925	75	92.5
Full context	8k/4k	350	650	35.0
Sliding window		534	466	53.4
LLM summary		784	216	78.4
Structured state		688	312	68.8
RAG memory		775	225	77.5
ARC		932	68	93.2
Full context	16k/8k	396	604	39.6
Sliding window		571	429	57.1
LLM summary		733	267	73.3
Structured state		688	312	68.8
RAG memory		788	212	78.8
ARC		991	9	99.1
Full context	32k/16k	373	627	37.3
Sliding window		543	457	54.3
LLM summary		759	241	75.9
Structured state		699	301	69.9
RAG memory		793	207	79.3
ARC		994	6	99.4

Table 7: Needle-in-a-Haystack evaluation on Qwen3-8B (16k context window, 4096 max-tokens) under different recall budgets. Each setting is evaluated on 1000 tasks.

HBM (High-Bandwidth Memory) Bandwidth Analysis

1. Experimental setup. All HBM roofline results in this ablation section are obtained by replaying the recorded LLM trajectories of each method on a single NVIDIA H200_141GB_SXM. We instantiate the hardware model with **141.0 GB** of HBM capacity, **4.8 TB/s** peak HBM bandwidth, and **989.0 TFLOPS** dense BF16 Tensor Core throughput. Because the experiments use one GPU only ($n_{\text{gpu}} = 1$), the aggregate hardware terms in the roofline model are exactly these per-device values.

The analysis scores each method on three quantities: total **decode HBM traffic**, **prefill compute time**, and their sum, i.e., the **roofline wall-clock estimate**. All methods are evaluated from the same trajectory logs, so they share the same model weights and hardware constants; differences arise from how each method changes the amount of live context that must be retained and reread across turns.

2. Roofline model. For each LLM call k , let p_k denote the prompt length and m_k the number of generated tokens. During autoregressive decode, each generated token streams

Model	Method	RECALL_BUDGET_CHARS	RECALL_MAX_CHARS	Success	Fail	Accuracy (%)
Qwen3-8B (16k context)						
	Full_context	8k	4k	0	311	0.00
	Sliding_window	8k	4k	79	232	25.40
	LLM_summary	8k	4k	83	228	26.69
	Structured_state	8k	4k	84	227	27.01
	RAG_memory	8k	4k	80	231	25.72
	ARC	8k	4k	81	230	26.05
	Full_context	16k	8k	0	311	0.00
	Sliding_window	16k	8k	77	234	24.76
	LLM_summary	16k	8k	83	228	26.69
	Structured_state	16k	8k	81	230	26.05
	RAG_memory	16k	8k	76	235	24.44
	ARC	16k	8k	91	220	29.26
	Full_context	32k	16k	0	311	0.00
	Sliding_window	32k	16k	76	235	24.44
	LLM_summary	32k	16k	72	239	23.15
	Structured_state	32k	16k	66	245	21.22
	RAG_memory	32k	16k	77	234	24.76
	ARC	32k	16k	85	226	27.33
Qwen3-32B (32k context)						
	Full_context	8k	4k	0	311	0.00
	Sliding_window	8k	4k	97	214	31.19
	LLM_summary	8k	4k	77	234	24.76
	Structured_state	8k	4k	91	220	29.26
	RAG_memory	8k	4k	85	226	27.33
	ARC	8k	4k	93	218	29.90
	Full_context	16k	8k	0	311	0.00
	Sliding_window	16k	8k	96	215	30.87
	LLM_summary	16k	8k	70	241	22.51
	Structured_state	16k	8k	93	218	29.90
	RAG_memory	16k	8k	88	223	28.30
	ARC	16k	8k	106	205	34.08
	Full_context	32k	16k	0	311	0.00
	Sliding_window	32k	16k	92	219	29.58
	LLM_summary	32k	16k	75	236	24.12
	Structured_state	32k	16k	92	219	29.58
	RAG_memory	32k	16k	80	231	25.72
	ARC	32k	16k	105	206	33.76

Table 8: LongBench results across methods under different recall budgets. Accuracy is computed as Success / (Success + Fail).

two components through HBM: (i) the model weights M_w , amortized across the decode batch size B ; and (ii) the request’s KV cache, whose cost scales with the active context length.

The per-token decode traffic is therefore

$$\text{bytes/token} = M_w/B + L_k \times kv_b, \quad (80)$$

where kv_b is the KV-cache footprint per token and L_k is the mean context length seen during decode for call k . Following the implementation, we approximate the mid-decode context by

$$L_k = p_k + (m_k - 1)/2. \quad (81)$$

Summing over all decode tokens in one trajectory yields total decode traffic

$$\text{BW}_{\text{decode}} = \sum_k m_k (M_w/B + L_k \times kv_b). \quad (82)$$

The achievable decode batch size is constrained by the remaining HBM capacity after reserving space for model weights and runtime scratch buffers:

$$B = \max \left(1, \left\lfloor \frac{\text{HBM} - M_w - \text{scratch}}{\bar{L} \times kv_b} \right\rfloor \right), \quad (83)$$

where $\bar{L}$ is the decode-token-weighted mean context length over the trajectory. This makes HBM capacity relevant even when the target metric is bandwidth: a larger KV budget permits a larger B , which reduces the amortized weight-read term M_w/B .

Decode time is then estimated by dividing total streamed bytes by peak HBM bandwidth:

$$T_{\text{decode}} = \text{BW}_{\text{decode}} / 4.8 \text{ TB/s}. \quad (84)$$

Prefill is modeled separately as compute-bound:

$$T_{\text{prefill}} = \text{FLOPs}_{\text{prefill}} / 989.0 \text{ TFLOPS}. \quad (85)$$

The combined roofline estimate reported in the table is

$$T_{\text{total}} = T_{\text{decode}} + T_{\text{prefill}}. \quad (86)$$

3. Why context management changes HBM cost. Under this model, context management affects HBM cost through two coupled mechanisms. First, shortening the live prompt directly reduces KV-cache traffic, since the term $L_k \times kv_b$ is paid for every generated token. Second, shorter contexts allow more requests to fit in the fixed 141.0 GB HBM budget, which increases B and further reduces the amortized weight-streaming term M_w/B . As a result, methods that keep less verbatim context active during multi-turn interaction can improve both **total HBM traffic** and **roofline decode time** on the same hardware.

Because the same Qwen3-8B model and the same H200 device are used for all methods, the measured differences in Table 9 should be interpreted as differences in **memory pressure induced by the context-management policy**, rather than as differences in underlying model architecture or hardware allocation.

4. Empirical trend on a single H200. Table 9 shows that **ARC is consistently the most HBM-efficient method** across all recall-budget settings. Relative to the SLIDING_WINDOW baseline, ARC reduces total decode traffic from **30.31 TB to 9.85 TB (66.0% savings)** at 4k / 2k, from **32.82 TB to 10.48 TB (66.4% savings)** at 8k / 4k, from **27.70 TB to 4.87 TB (80.3% savings)** at 16k / 8k, and from **31.85 TB to 4.93 TB (82.6% savings)** at 32k / 16k.

These bandwidth reductions translate into substantial roofline speedups: **2.94×**, **2.97×**, **5.08×**, and **5.75×** over SLIDING_WINDOW, respectively. ARC also attains the highest success count in every setting (925, 932, 991, and 994), indicating that its lower HBM cost is not obtained by sacrificing task effectiveness.

The remaining baselines are notably weaker on this hardware. RAG_MEMORY is the closest competitor in absolute bandwidth, but its savings are modest and inconsistent (6.7%, 8.4%, −1.4%, and 13.3%). LLM_SUMMARY and STRUCTURED_STATE are above SLIDING_WINDOW in total HBM traffic for all four budget settings. Overall, the single-H200 analysis shows that ARC’s main systems advantage is not only better task success, but a much smaller amount of HBM traffic per solved trajectory, with the advantage widening as the recall budget becomes less restrictive.

The forced-compaction ablation exhibits the same qualitative trend as the recall-budget study: **ARC remains the most HBM-efficient method under every compaction schedule**. In Table 10, under the default 5, 10, 15 schedule, ARC reduces per-success decode traffic from **27.70 TB to 4.87 TB (80.3% reduction)** and decreases roofline time from **6.378 s to 1.256 s (5.08×** speedup) relative to SLIDING_WINDOW, while also achieving the highest number of successful tasks (**991** versus **571**). The same pattern holds when compaction is forced only at 5 or only at 15: ARC again achieves the lowest bandwidth consumption (**5.12 TB** and **6.82 TB** per successful task, respectively), the lowest per-success execution time (**1.314 s** and **1.709 s**), and the highest success counts (**998** and **997**).

The strongest contrast appears under the most aggressive schedule, 3, 6, 9, 12, 15. Here, the SLIDING_WINDOW baseline succeeds on only **189** tasks, whereas ARC succeeds on **989**. Because the reported metrics are normalized *per successful task*, this substantial difference in the denominator makes the non-ARC baselines appear particularly expensive on cost-per-success metrics. Even with this caveat, the overall trend remains unchanged: ARC still achieves the smallest bandwidth footprint (**4.96 TB per success**) and the lowest latency (**1.290 s per success**), while LLM_SUMMARY, STRUCTURED_STATE, and RAG_MEMORY all remain substantially more expensive. Overall, the forced-compaction results demonstrate that ARC’s HBM efficiency is robust not only to the recall budget, but also to the frequency with which compaction is triggered.

HBM for the Needle-in-a-Haystack Ablation: Fixture size (noise_lines_per_side) The fixture-size ablation (Table 11) shows that **ARC remains the most HBM-efficient method as the distractor observation becomes larger**.

Method	RECALL (BUDGET_CHARS/MAX_CHARS)	n _{succ} ↑	t _{tot} ↓ (s)	t _{dec} ↓ (s)	t _{pre} ↓ (s)	BW ↓ (TB)	Savings (%) ↑	Speedup ↑ (×)
Sliding Window	4k / 2k	570	6.970	6.316	0.655	30.3146	0.0	1.00
LLM Summary		762	8.619	8.112	0.507	38.9376	-23.6	0.81
Structured State		730	9.336	8.897	0.438	42.7060	-33.9	0.75
RAG Memory		800	6.503	6.067	0.436	29.1240	6.7	1.07
ARC		925	2.367	2.051	0.316	9.8453	66.0	2.94
Sliding Window	8k / 4k	534	7.512	6.837	0.675	32.8164	0.0	1.00
LLM Summary		784	8.380	7.879	0.501	37.8189	-11.6	0.90
Structured State		688	10.132	9.675	0.457	46.4407	-34.9	0.74
RAG Memory		775	6.878	6.424	0.454	30.8357	8.4	1.09
ARC		932	2.527	2.184	0.343	10.4825	66.4	2.97
Sliding Window	16k / 8k	571	6.378	5.771	0.607	27.7007	0.0	1.00
LLM Summary		733	9.494	8.952	0.543	42.9673	-48.9	0.67
Structured State		688	10.589	10.115	0.474	48.5509	-66.0	0.60
RAG Memory		788	6.467	6.025	0.442	28.9208	-1.4	0.99
ARC		991	1.256	1.014	0.241	4.8694	80.3	5.08
Sliding Window	32k / 16k	543	7.294	6.636	0.658	31.8519	0.0	1.00
LLM Summary		759	8.925	8.414	0.511	40.3860	-22.4	0.82
Structured State		699	9.910	9.456	0.454	45.3879	-35.9	0.74
RAG Memory		793	6.325	5.890	0.435	28.2742	13.3	1.15
ARC		994	1.269	1.028	0.242	4.9322	82.6	5.75

Table 9: HBM evaluation per successful trajectory for different methods under varying recall budgets. The baseline for savings and speedup is SLIDING_WINDOW within each recall budget.

Method	FORCE	n _{succ} ↑	t _{tot} ↓ (s)	t _{dec} ↓ (s)	t _{pre} ↓ (s)	BW ↓ (TB)	Savings (%) ↑	Speedup ↑ (×)
Sliding window	5, 10, 15	571	6.378	5.771	0.607	27.7007	0.0	1.00
LLM summary		733	9.494	8.952	0.543	42.9673	-48.9	0.67
Structured state		688	10.589	10.115	0.474	48.5509	-66.0	0.60
RAG memory		788	6.467	6.025	0.442	28.9208	-1.4	0.99
ARC		991	1.256	1.014	0.241	4.8694	80.3	5.08
Sliding window	5	666	5.174	4.613	0.561	22.1434	0.0	1.00
LLM summary		763	8.617	7.996	0.621	38.3788	-66.5	0.60
Structured state		769	10.431	9.928	0.504	47.6524	-101.6	0.50
RAG memory		799	7.397	6.926	0.472	33.2431	-43.0	0.70
ARC		998	1.314	1.066	0.248	5.1184	74.6	3.94
Sliding window	15	673	5.701	4.922	0.778	23.6278	0.0	1.00
LLM summary		765	9.470	8.667	0.803	41.6012	-66.1	0.60
Structured state		798	8.948	8.185	0.763	39.2882	-57.0	0.64
RAG memory		790	6.856	6.327	0.529	30.3697	-20.3	0.83
ARC		997	1.709	1.421	0.288	6.8212	70.0	3.34
Sliding window	3, 6, 9, 12, 15	189	2.200	2.115	0.085	10.1509	0.0	1.00
LLM summary		876	4.944	4.508	0.436	21.6371	-124.7	0.45
Structured state		652	10.782	10.283	0.499	49.3566	-390.1	0.20
RAG memory		799	9.090	8.688	0.402	41.7001	-313.2	0.24
ARC		989	1.290	1.034	0.256	4.9618	41.4	1.71

Table 10: HBM evaluation per successful trajectory for different methods on Qwen3-8B (16k context window, 4096 max-tokens) under varying FORCE configurations. The baseline for savings and speedup computations is the SLIDING_WINDOW method within each respective configuration block.

Across all four settings (5, 10, 25, and 30 noise lines per side), ARC achieves the lowest total time, the lowest decode time, and the smallest HBM traffic, while also maintaining the highest success count. Relative to `SLIDING_WINDOW`, ARC reduces bandwidth from **30.40 TB** to **3.07 TB (88.8% savings)** at fixture size 5, from **25.93 TB** to **3.45 TB (84.9% savings)** at 10, from **27.70 TB** to **4.87 TB (80.3% savings)** at 25, and from **26.71 TB** to **5.09 TB (78.5% savings)** at 30.

These HBM savings translate into large roofline speedups throughout the sweep: ARC improves per-success total time from **6.845 s** to **0.769 s (8.90 $\times$)** at fixture size 5, from **5.871 s** to **0.885 s (6.63 $\times$)** at 10, from **6.378 s** to **1.256 s (5.08 $\times$)** at 25, and from **6.221 s** to **1.339 s (4.65 $\times$)** at 30. The absolute ARC bandwidth and latency do increase slightly as the fixture grows, which is expected because larger distractor observations create longer live contexts and therefore more KV-cache traffic. However, this degradation is much smaller for ARC than for the baselines, indicating that citation-based compaction continues to suppress HBM growth even when the retrieved observation itself becomes more expensive.

Among the non-ARC methods, `RAG_MEMORY` is again the closest competitor at the smallest fixture sizes, with **17.2%** savings at 5 and **34.4%** at 10, but it loses this advantage as the fixture grows and becomes worse than `SLIDING_WINDOW` at 25 and 30. `LLM_SUMMARY` and `STRUCTURED_STATE` are consistently above the `SLIDING_WINDOW` baseline in both bandwidth and total time. Overall, the fixture-size sweep shows that ARC’s HBM advantage is robust not only to recall budget and compaction schedule, but also to the size of the needle-containing observation that must be managed.

Cross-Platform Validation: AMD MI300X

To verify that ARC’s advantage is not an artifact of a single vendor’s hardware or numerical stack, we replicate the Needle-in-a-Haystack evaluation from the main paper on AMD Instinct MI300X GPUs (192 GB HBM3, running ROCm) using the identical experimental protocol: Qwen3-8B at a 16k context window and Qwen3-32B at a 32k context window, both under a 4,096-token completion cap, with the same three seeds (42, 82, 122).

Accuracy

Table 12 reports mean accuracy across the three seeds on AMD MI300X, directly comparable to Table 2 in the main paper (NVIDIA H200).

ARC’s accuracy is essentially unchanged across vendors: 99.00% (H200) versus 99.57% (MI300X) on Qwen3-8B, and 99.80% versus 99.77% on Qwen3-32B, in both cases within the seed-to-seed noise already present on a single platform. This indicates that ARC’s recovery guarantee holds independent of the underlying accelerator.

The baseline methods show a different pattern. At Qwen3-8B, every baseline scores substantially higher on MI300X than on H200 (e.g., `Full_context` rises from 36.80% to 53.07%, `LLM_summary` from 71.77% to 83.57%). At Qwen3-32B, this gap largely disappears: every method, baselines included, lands within roughly one point of its H200 counterpart. Because ARC’s own accuracy is stable across

both configurations, this discrepancy affects only the comparison among lossy baselines and does not bear on ARC’s central claim.

Hardware-Cost Model

We additionally replay each method’s recorded trajectory through the same roofline hardware-cost model used in the main paper (Eq. 1–2), substituting AMD MI300X specifications (192 GB HBM3, 5.3 TB/s peak bandwidth, 1300 TFLOPS BF16 dense compute) for the NVIDIA H200 figures. Following the same per-success normalization as the main paper, and at the request of a collaborator to compare against a compaction baseline rather than the non-compacting control, we report savings relative to *Sliding_window* rather than *Full_context*.

ARC is the fastest and lowest-bandwidth method at both scales. On Qwen3-8B, ARC completes each success in 1.01 s at 4.38 TB of HBM traffic, a 79.6% time saving relative to `Sliding_window`; every other lossy baseline is in fact *slower* than `Sliding_window` at this scale (Sav. is negative for `LLM_summary`, `Structured_state`, and `RAG_memory`), so ARC is the only method that both compacts and is faster than the naive windowing baseline it is compared against. On Qwen3-32B, ARC again leads on both time (1.57 s) and bandwidth (4.32 TB), though `Structured_state` narrows the gap on this larger model (65.4% savings versus ARC’s 73.8%).

Summary

These results indicate that ARC’s core contribution, exact, addressable recovery of compacted content, transfers across accelerator vendors without modification. The magnitude of ARC’s efficiency advantage over lossy baselines is consistent with the main paper’s NVIDIA findings, supporting the claim that ARC’s benefits stem from its recall mechanism rather than from vendor-specific kernel behavior.

Method	Noise Lines (per side)	n _{succ} ↑	t _{tot} ↓ (s)	t _{dec} ↓ (s)	t _{pre} ↓ (s)	BW ↓ (TB)	Savings (%) ↑	Speedup ↑ (×)
Sliding window	5	522	6.845	6.333	0.512	30.3991	0.0	1.00
LLM summary		738	8.824	8.438	0.386	40.5024	-28.9	0.78
Structured state		676	10.164	9.761	0.403	46.8541	-48.5	0.67
RAG memory		797	5.666	5.361	0.305	25.7347	17.2	1.21
ARC		999	0.769	0.639	0.130	3.0658	88.8	8.90
Sliding window	10	603	5.871	5.402	0.469	25.9292	0.0	1.00
LLM summary		790	7.894	7.516	0.378	36.0761	-34.4	0.74
Structured state		743	8.864	8.494	0.370	40.7730	-51.0	0.66
RAG memory		872	3.852	3.599	0.253	17.2744	34.4	1.52
ARC		1000	0.885	0.718	0.167	3.4457	84.9	6.63
Sliding window	25	571	6.378	5.771	0.607	27.7007	0.0	1.00
LLM summary		733	9.494	8.952	0.543	42.9673	-48.9	0.67
Structured state		688	10.589	10.115	0.474	48.5509	-66.0	0.60
RAG memory		788	6.467	6.025	0.442	28.9208	-1.4	0.99
ARC		991	1.256	1.014	0.241	4.8694	80.3	5.08
Sliding window	30	631	6.221	5.566	0.655	26.7149	0.0	1.00
LLM summary		750	10.109	9.523	0.586	45.7103	-62.5	0.62
Structured state		731	10.083	9.615	0.467	46.1528	-62.1	0.62
RAG memory		755	7.805	7.292	0.513	35.0015	-25.5	0.80
ARC		992	1.339	1.060	0.279	5.0904	78.5	4.65

Table 11: Evaluation results per successful trajectory for various memory mechanisms on Qwen3-8B (16k context window, 4096 max-tokens) under different structural noise conditions (NOISE_LINES_PER_SIDE).

Method	Pass/Att.	Mean Acc↑	SD Acc
<i>Qwen3-8B</i>			
Full_context	1592/3000	53.07%	1.45%
Sliding_window	2055/3000	68.50%	0.35%
LLM_summary	2507/3000	83.57%	1.99%
Structured_state	2355/3000	78.50%	0.89%
RAG_memory	2431/3000	81.03%	1.46%
ARC	2987/3000	99.57%	0.35%
<i>Qwen3-32B</i>			
Full_context	118/3000	3.93%	1.08%
Sliding_window	2863/3000	95.43%	0.35%
LLM_summary	2815/3000	93.83%	0.68%
Structured_state	2882/3000	96.07%	0.59%
RAG_memory	2925/3000	97.50%	0.17%
ARC	2993/3000	99.77%	0.23%

Table 12: Needle-in-a-Haystack accuracy on AMD MI300X, 3 seeds (42, 82, 122) of 1000 tasks each (3000 attempts/method). Directly comparable to Table 2 (NVIDIA H200) in the main paper.

Method	t _{tot} .(s)↓	bw(TB)↓	Sav.(%)↑
<i>Qwen3-8B</i>			
Sliding_window	4.942	23.92	0.0
LLM_summary	6.293	31.45	-27.3
Structured_state	7.853	39.99	-58.9
RAG_memory	5.134	25.56	-3.9
ARC	1.009	4.38	79.6
<i>Qwen3-32B</i>			
Sliding_window	6.010	26.79	0.0
LLM_summary	3.859	16.83	35.8
Structured_state	2.079	8.18	65.4
RAG_memory	3.462	15.29	42.4
ARC	1.572	4.32	73.8

Table 13: High-bandwidth-memory comparison on AMD MI300X, Needle-in-a-Haystack, same 4,096-token completion cap as the main paper (seed 42). Savings reported relative to Sliding_window, per the request of a project collaborator to compare against a compaction-based baseline rather than the non-compacting control.